

modularity

Daniel Jackson

what you'll learn today

how to design modular software
so much more than “use interfaces”

where we are

defining a concept

concept Inviting

purpose ensure mutual consent for participation in some engagement; prevents unwanted participation.

principle A host invites guests one at a time to participate in some engagement. Each guest can accept or decline the invitation, until the invitation expires after which no responses are accepted.

state

a set of Invitations with
an Engagement
an invited Guest
an expiry Date
a status of
PENDING, ACCEPTED, DECLINED, EXPIRED

actions

invite (guest: Guest, engagement: Engagement)

where no invitation for guest and engagement

then

create new invitation for guest and engagement
with expiry of one month from now
and status PENDING

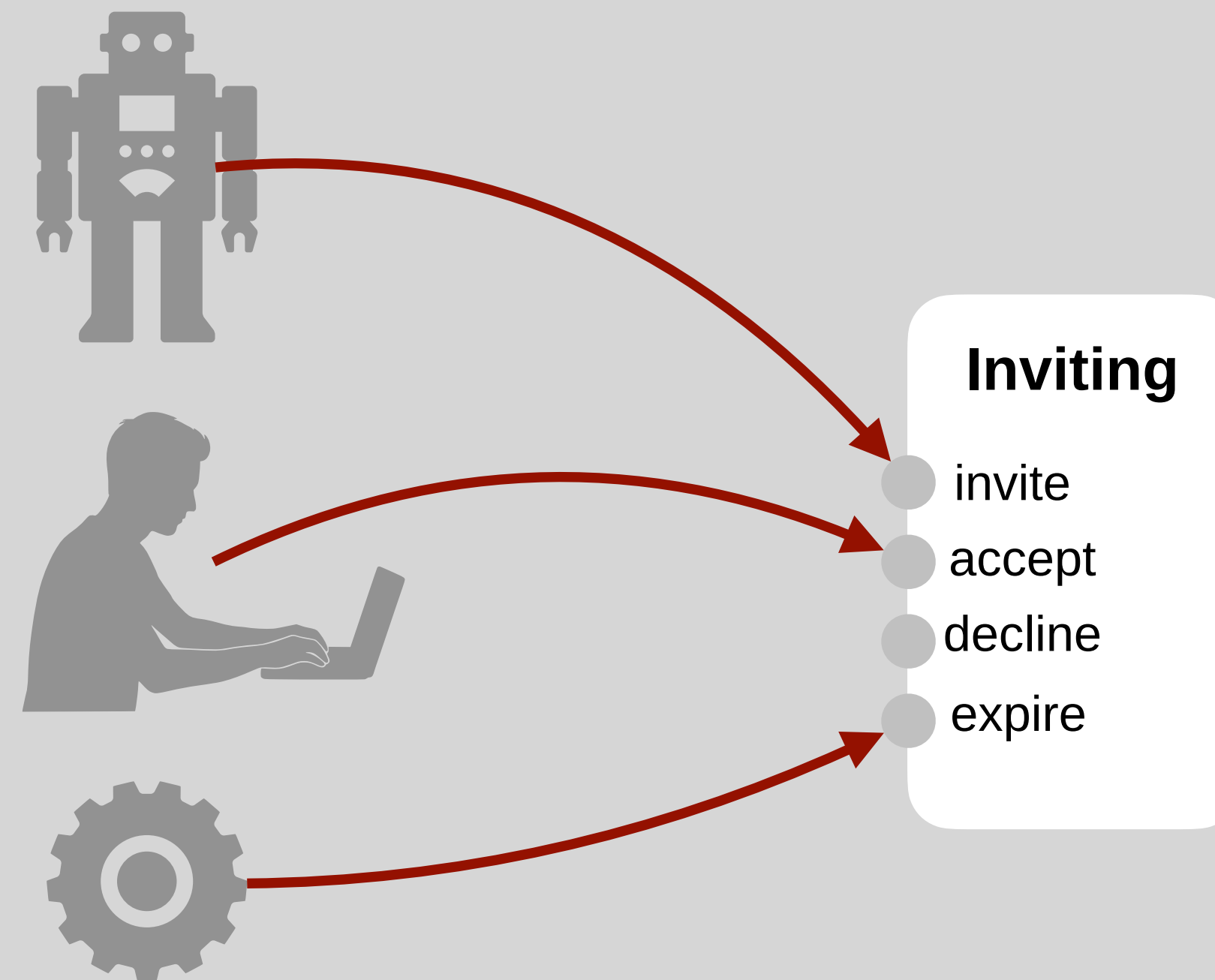
accept (guest: Guest, engagement: Engagement)

where some invitation for guest and engagement
with status PENDING

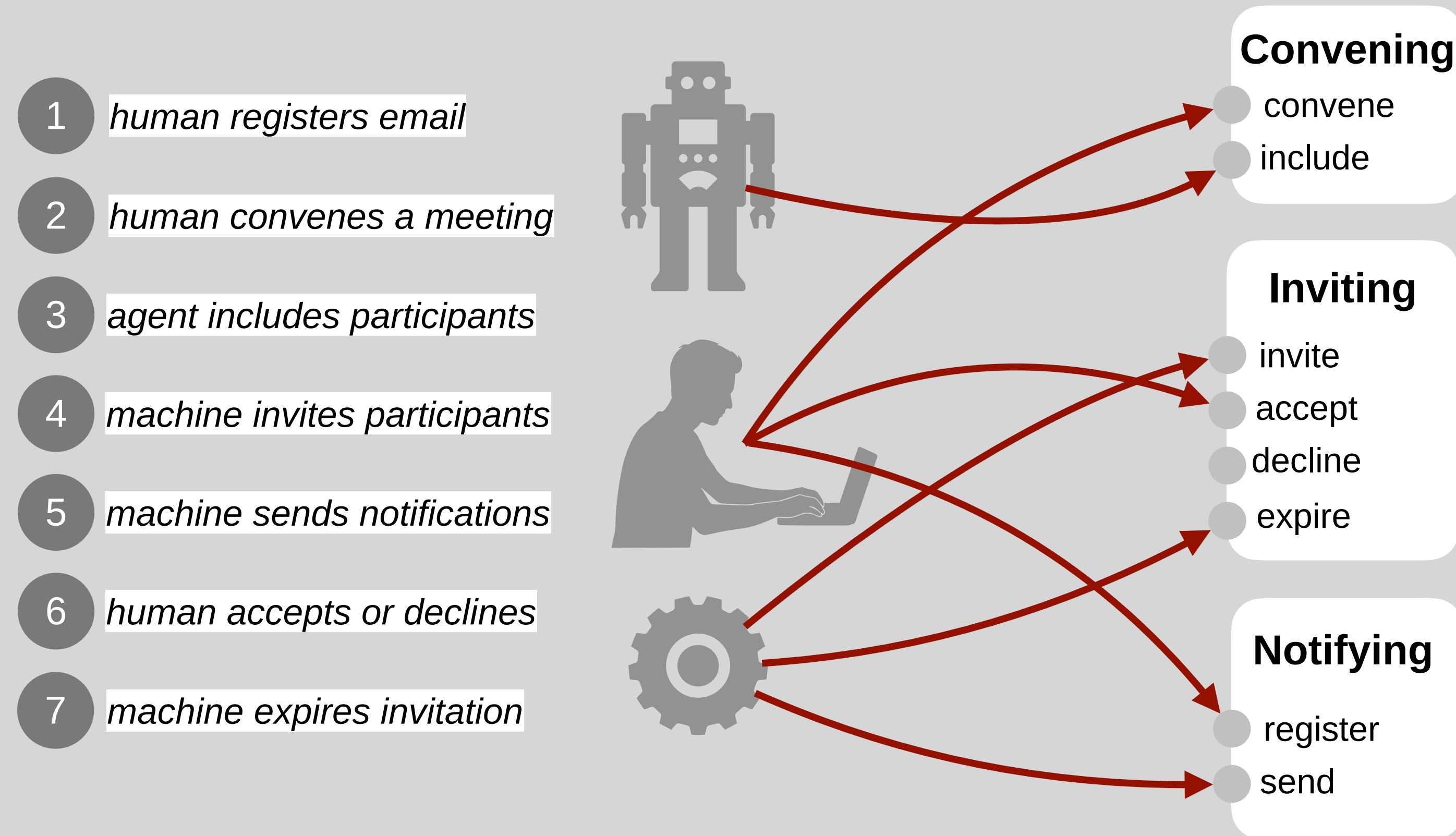
then set status of invitation to ACCEPTED

...

from design language to realization



composing concepts into a system



how reactions are specified

when

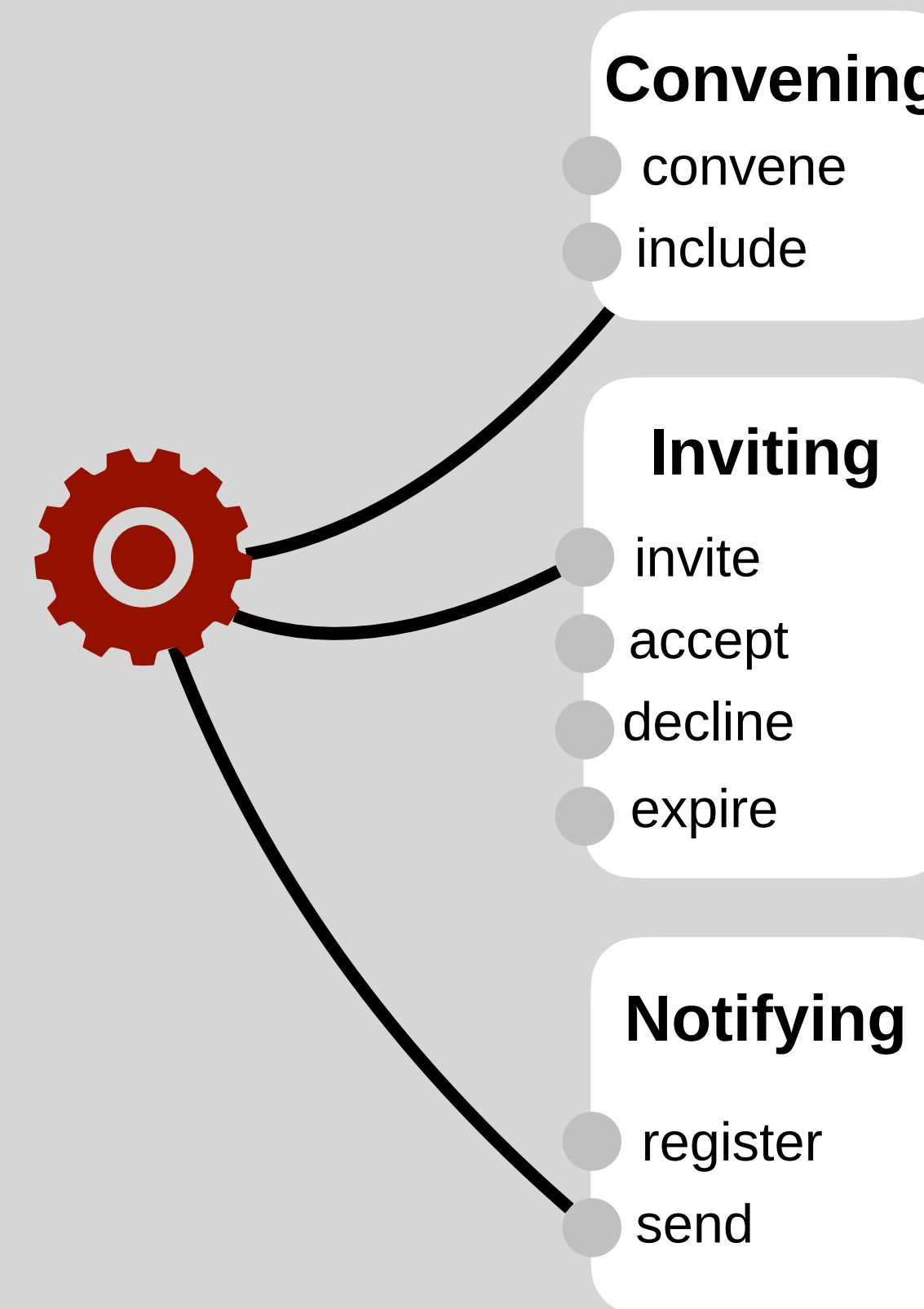
Inviting.invite (guest, engagement)

where

Convening: engagement has title and date

then

Notifying.send (user: guest, msg:
"Please come to " + engagement +
" on " + date)



this is called "event-driven programming"
it's a common paradigm in large scale systems

what do you think are
its benefits and costs?

concepts for
hacker news

▲ Jackson structured programming (wikipedia.org)
106 points by haakonhr 63 days ago | hide | past | favorite | 69 comments

Posting

Sessioning

Upvoting

Favoriting

▲ danielnicholas 63 days ago [-]
user: danielnicholas
created: 63 days ago
karma: 11

Commenting

KarmaTracking

you might find helpful an annotated version [0] of Hoare's explanation of JSP that I edited for a Michael Jackson festschrift

I'd point to these ideas as worth knowing:

...ing problem that involves traversing ...res can be solved very systematically. HTDP addresses this class, but bases ...ue structure only on input structure; JSP synthesized i

metypal problems that, however you code, can't be pushed under the rug—most notably structure clashes—and just recognizing them helps.

- Coroutines (or code transformation) let you structure code more cleanly when you need to read or write more than one structure. It's why real iterators (with yield), which offer a limited form of this, are (in my view) better than Java-style iterators with a next method.
- The idea of viewing a system as a collection of asynchronous processes (Ch. 11 in the JSP book, which later became JSD) with a long-running process for each real-world entity. This was a notable contrast to OOP, and led to a strategy (seeing a resurgence with event storming for DDD) that began with events rather than objects.

[0] <https://groups.csail.mit.edu/sdg/pubs/2009/hoare-jsp-3-29-09...>

▲ ob-nix 63 days ago [-]

... this brings back memories! In the late eighties I, as a teenager, found a Jackson Struct. Pr. book at the town library. I remember I was amazed at the text and wondered why I hadn't heard about the method before.

If I remember correctly did the book clearly point out backtracking as a standard method, while mentioning that most languages lacked that, so it had to be implemented manually.

▲ CraigJPerry 63 days ago [-]

This is referenced(1) as a core inspiration in the preface to "How to Design Programs" but i never researched it further because i've found the "design recipes" approach in htdp to be pretty solid in real life problems

a sample concept: Upvoting

Y **Hacker News** new | past | comments | ask | show

▲ GPT-5 (openai.com)

2929 points by rd 22 hours ago | hide | past | favorite | 2298 comments

https://www.youtube.com/watch?v=0Uu_VJeVVfo



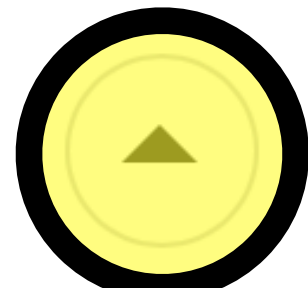
One more for dinner

Charlottesville | 3h ago

Again, please. He didn't take this position for our benefit. He did it for his.

[Reply](#) [65 Recommend](#) [Share](#)

[Flag](#)



4



Can any one here explains how the transitive closure operator works in Alloy in terms of the matrix. I mean what's translation rule for translating closure operator into actual matrix operation.

alloy

what's a concept?

a coherent **course of action**

realized as a mechanism

reusable & familiar

designed, coded and explained **independently**

defining the concept

concept Upvoting [User, Item]

purpose rank items by popularity

principle after a series of upvotes, can rank items by the number of votes they received

state

a set of Votes with
a voter User
a target Item

actions

upvote (user: User, item: Item)
unvote (user: User, item: Item)

new to concept design

standard computer science

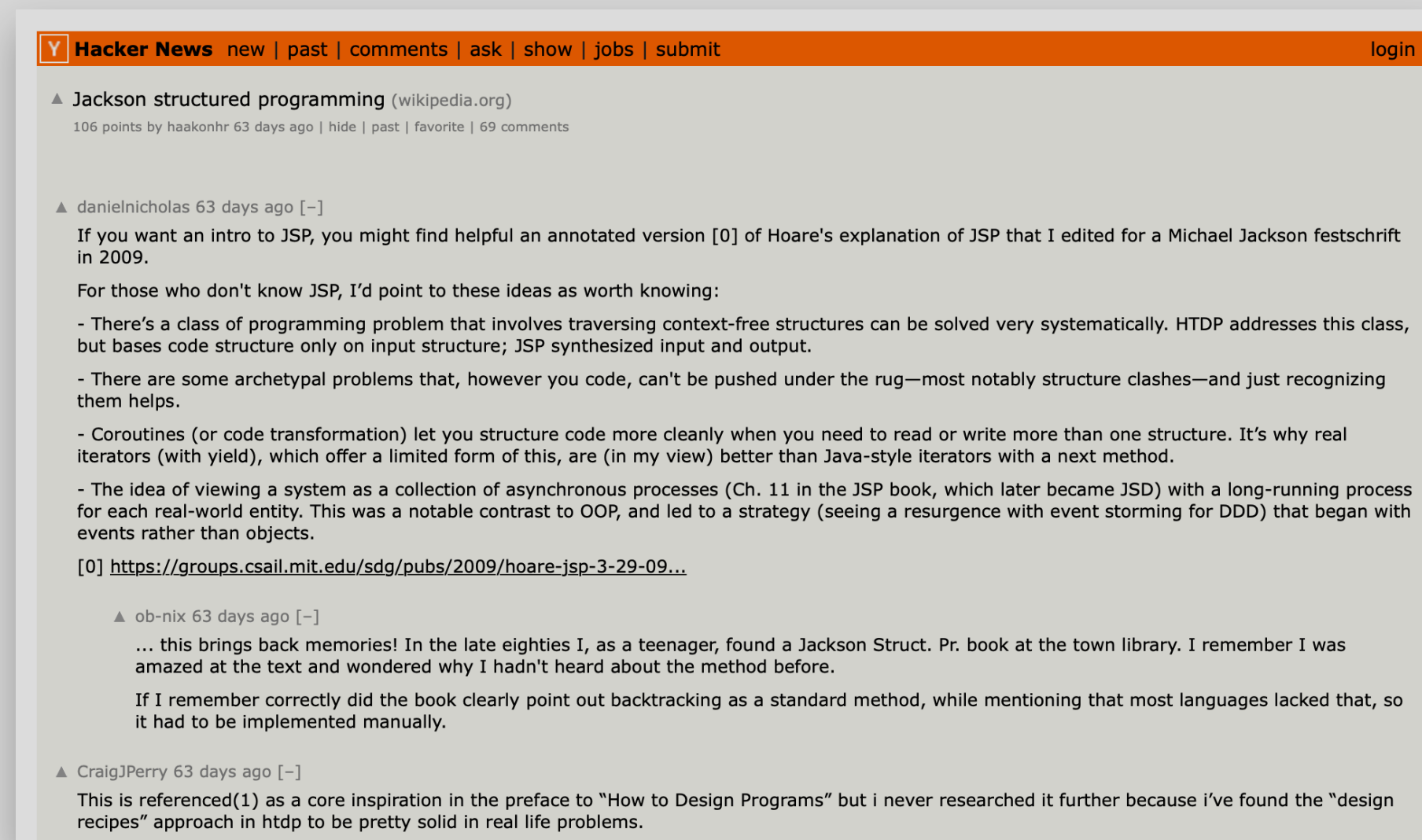
what bad situation
does it prevent?

what's an invariant of
the state?

how would you add
downvoting?

why the unvote
action?

how unique is it?



no other app is the same as HackerNews

but its concepts are mostly identical to the concepts in other apps:

HackerNews = Posting + Commenting + Upvoting + KarmaTracking + ...

so where's the
innovation?

so where's the innovation?

▲ Jackson structured programming
106 points by haakonhr 63 days ago

hacker news is popular

> 10m page views/day

so evidently it was worth building

where's the innovation?

“combinational creativity” [Boden]

familiar elements combined in new ways

innovations in concepts themselves

in Posting, post is title + link

innovations in concept coordination

no downvote until your own posts upvoted

▲ danielnicholas 63 days ago [-]

If you want an intro to JSP, see this post from 2009.

For those who don't know JSP, here's a quick overview.

- There's a class of programming language called JSP, but bases code structure on Java.

- There are some archetypes of JSP, and knowing them helps.

- Coroutines (or code transformation with yield), which is a JSP feature.

- The idea of viewing a system as a sequence of events for each real-world entity. This is a JSP feature, and events rather than objects.

[0] <https://groups.csail.mit.edu/tech/ai/mlg/2004fall/reading/10-jackson-structured-programming/>

▲ ob-nix 63 days ago [-]

... this brings back memories! In the late eighties I, as a teenager, found a Jackson Struct. Pt. book at the town library. I remember I was amazed at the text and wondered why I hadn't heard about the method before.

If I remember correctly did the book clearly point out backtracking as a standard method, while mentioning that most languages lacked that, so it had to be implemented manually.

a Michael Jackson festschrift

ly. HTDP addresses this class,

ishes—and just recognizing

structure. It's why real
od.

) with a long-running process
ning for DDD) that began with

concepts as carriers of design knowledge

concept: Upvoting

related concepts

Rating, Recommending, Reacting, ...

design variants

downvote as unvote
use age in ranking
weigh downvotes more
various identity tactics
freezing old posts

typical uses

social media posts
comments on articles
Q&A responses



often used with

KarmaTracking, Authenticating ...

what are some known issues?

known issues

high votes can promote old content
feedback favors early upvotes
upvoting encourages echo chamber
preventing double votes

extending
hacker news

▲ Jackson structured programming (wikipedia.org)

106 points by haakonhr 63 days ago | hide | past | favorite | 69 comments

▲ danielnicholas 63 days ago [-]

If you want an intro to JSP, you might find helpful an annotated version [0] of Hoare's explanation of JSP that I edited for a Michael Jackson festschrift in 2009.

For those who don't know JSP, I'd point to these ideas as worth knowing:

- There's a class of programming problem that involves traversing context-free structures can be solved very systematically. HTDP addresses this class, but bases code structure only on input structure; JSP synthesized input and output.
- There are some archetypal problems that, however you code, can't be pushed under the rug—most notably structure clashes—and just recognizing them helps.
- Coroutines (or code transformation) let you structure code more cleanly when you need to read or write more than one structure. It's why real iterators (with yield), which offer a limited form of this, are (in my view) better than Java-style iterators with a next method.
- The idea of viewing a system as a collection of asynchronous processes (Ch. 11 in the JSP book, which later became JSD) with a long-running process for each real-world entity. This was a notable contrast to OOP, and led to a strategy (seeing a resurgence with event storming for DDD) that began with events rather than objects.

[0] <https://groups.csail.mit.edu/sdg/pubs/2009/hoare-jsp-3-29-09...>

▲ ob-nix 63 days ago [-]

... this brings back memories! In the late eighties I, as a teenager, found a Jackson Struct. Pr. book at the town library. I remember I was amazed at the text and wondered why I hadn't heard about the method before.

If I remember correctly did the book clearly point out backtracking as a standard method, while mentioning that most languages lacked that, so it had to be implemented manually.

adding application-specific functionality

concept Upvoting

purpose rank items by popularity

actions

upvote (user, item)

downvote (user, item)

unvote (user, item)

suppose I want this behavior:

you can't downvote a post
until you've received
two upvotes on your own posts

could just modify Upvoting

why is this bad?

define a new concept!

a hint: not just used by Upvoting

concept Posting

purpose share content

state

a set of Posts with

a body String

an author User

actions

create (user, body): post

delete (post)

edit (post, body)

concept KarmaTracking

purpose privilege good users

state

a set of Users with

a karma Number

actions

reward (user, reward)

a first reaction

concept Upvoting

actions

upvote (user, item)
downvote (user, item)
unvote (user, item)

when Upvoting.upvote (post)
where Posting: author of post is user
then KarmaTracking.reward (user, 10)

concept Posting

state

a set of Posts with
a body String
an author User

actions

create (user, body): post
delete (post)
edit (post, body)

concept KarmaTracking

state

a set of Users with
a karma Number

actions

reward (user, reward)

a second reaction

concept Upvoting

actions

upvote (user, item)
downvote (user, item)
unvote (user, item)

concept Requesting

actions

request (action)
respond (response)

when Requesting.request (
 downvote, user, post)
where KarmaTracking: user karma $\geq$ 20
then Upvoting.downvote (user, post)

concept Posting

state

a set of Posts with
 a body String
 an author User

actions

create (user, body): post
delete (post)
edit (post, body)

concept KarmaTracking

state

a set of Users with
 a karma Number

actions

reward (user, reward)

controlling downvoting in two reactions

concept Upvoting

actions

upvote (user, item)
downvote (user, item)
unvote (user, item)

concept Requesting

actions

request (action)
respond (response)

when Upvoting.upvote (post)
where Posting: author of post is user
then KarmaTracking.reward (user, 10)

when Requesting.request (
downvote, user, post)
where KarmaTracking: user karma $\geq$ 20
then Upvoting.downvote (user, post)

why not upvote (user, post) in first reaction?

concept Posting

state

a set of Posts with
a body String
an author User

actions

create (user, body): post
delete (post)
edit (post, body)

concept KarmaTracking

state

a set of Users with
a karma Number

actions

reward (user, reward)

reactions viewed over traces

concept Requesting

concept Upvoting

concept KarmaTracking

concept Posting

create (Alice, ...) : p1

create (Bob, ...) : p2

upvote (Bob, p1)

not a **call**
concepts are
decoupled

reward (Alice, 10)

author of p1 is Alice

upvote (Carol, p1)

reward (Alice, 10)

author of p1 is Alice

request(downvote, Alice, p2)

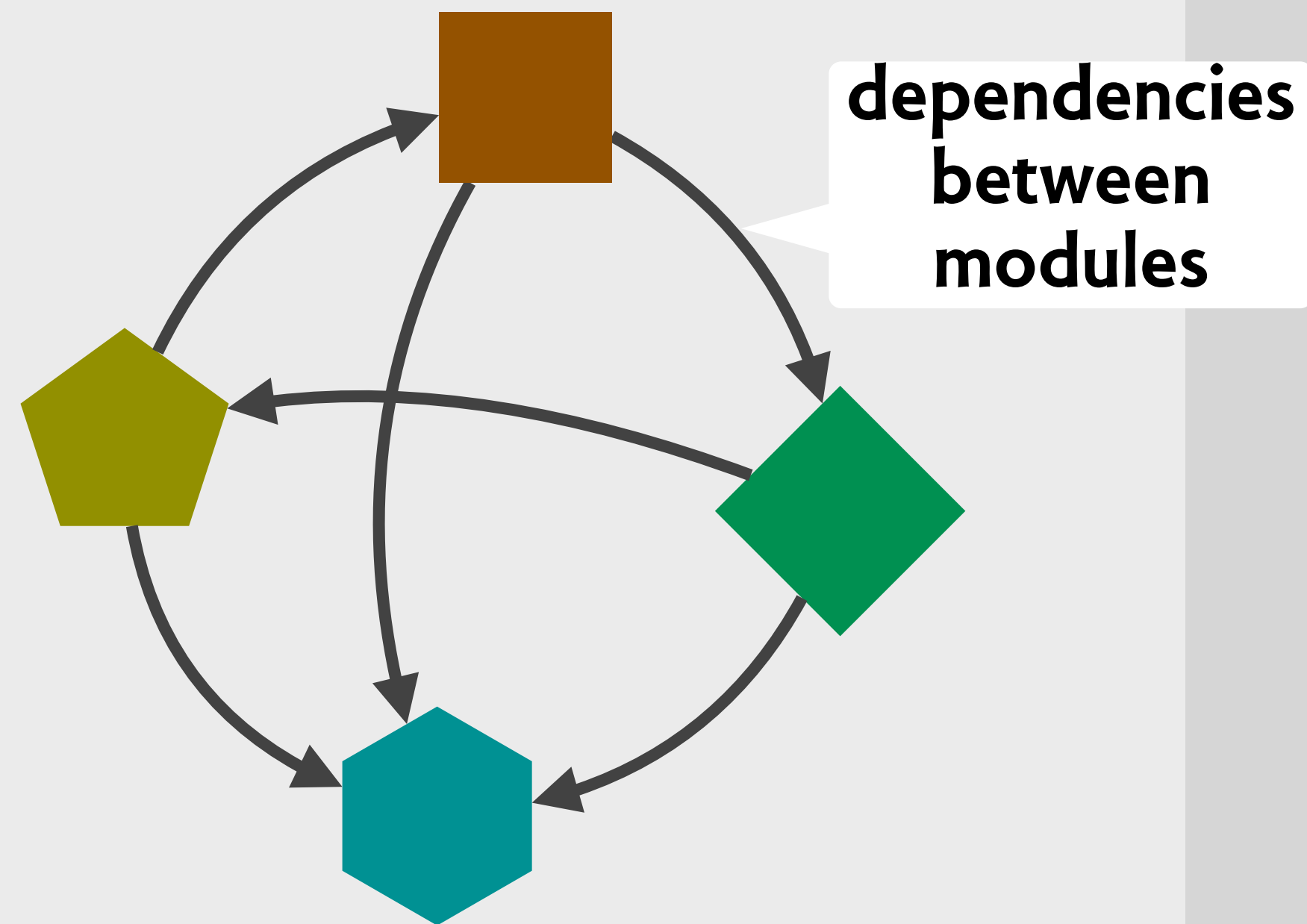
downvote (Alice, p2)

Alice has ≥ 20 karma

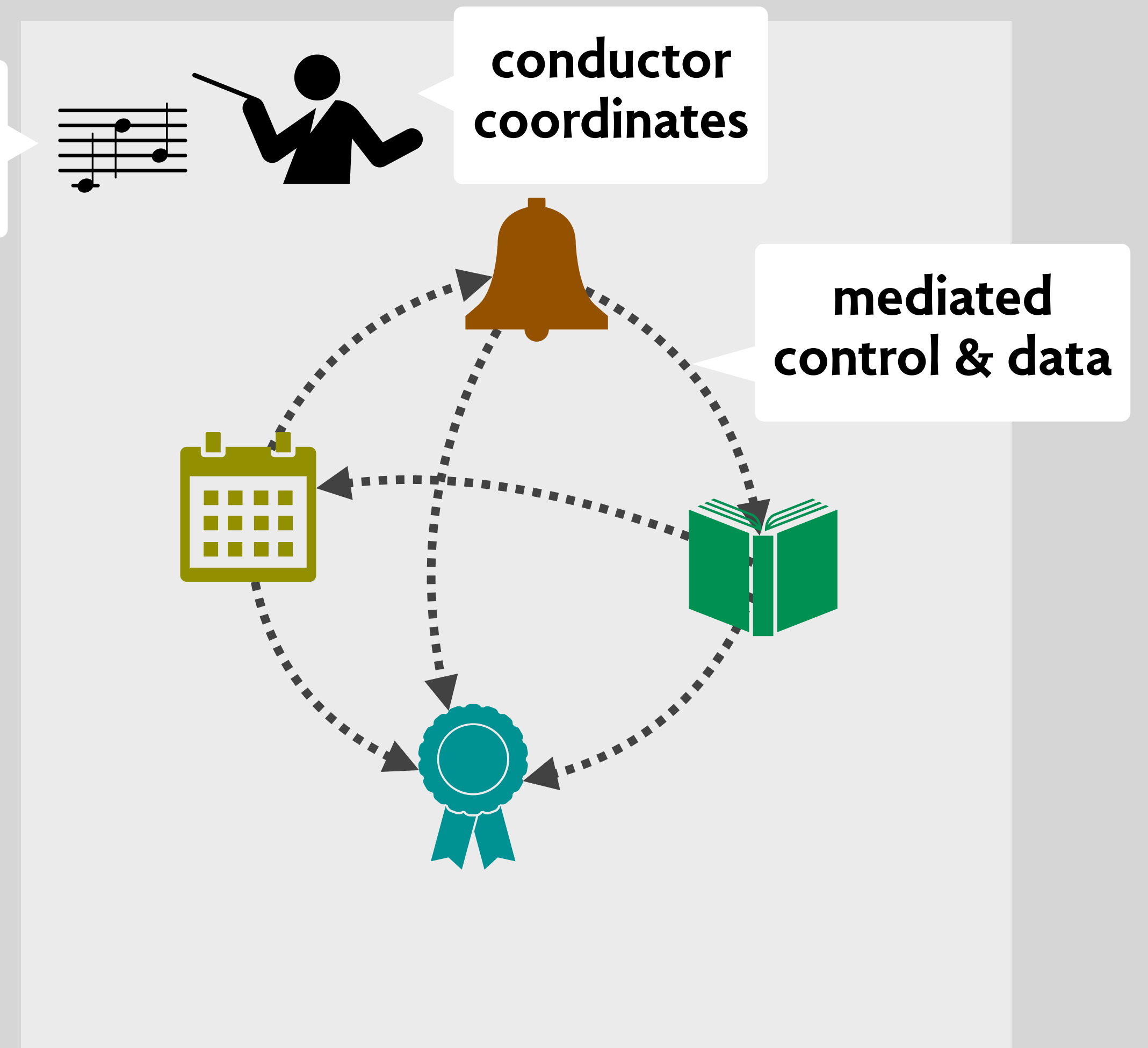
concepts stay
largely
application
independent

each concept
executes a
valid scenario

an architectural view: externalizing connections



1980s software development



concept-based software development

a reminder: how we didn't do it

Upvoting

*upvote
reads
author
from Posting
and calls
reward in
KarmaTracking*

concepts never
call each other's actions
read or write each other's state
share mutable composite objects

your turn

more karma

can you think of other ways
to use karma in a social media app?

some ideas

improve the reliability of upvoting?
counter botnet attacks?
reward star contributors?

your task

talk to the person next to you
come up with some ideas
submit an idea each

for each idea

which concepts?
what reactions?

Lecture 5 (karma)

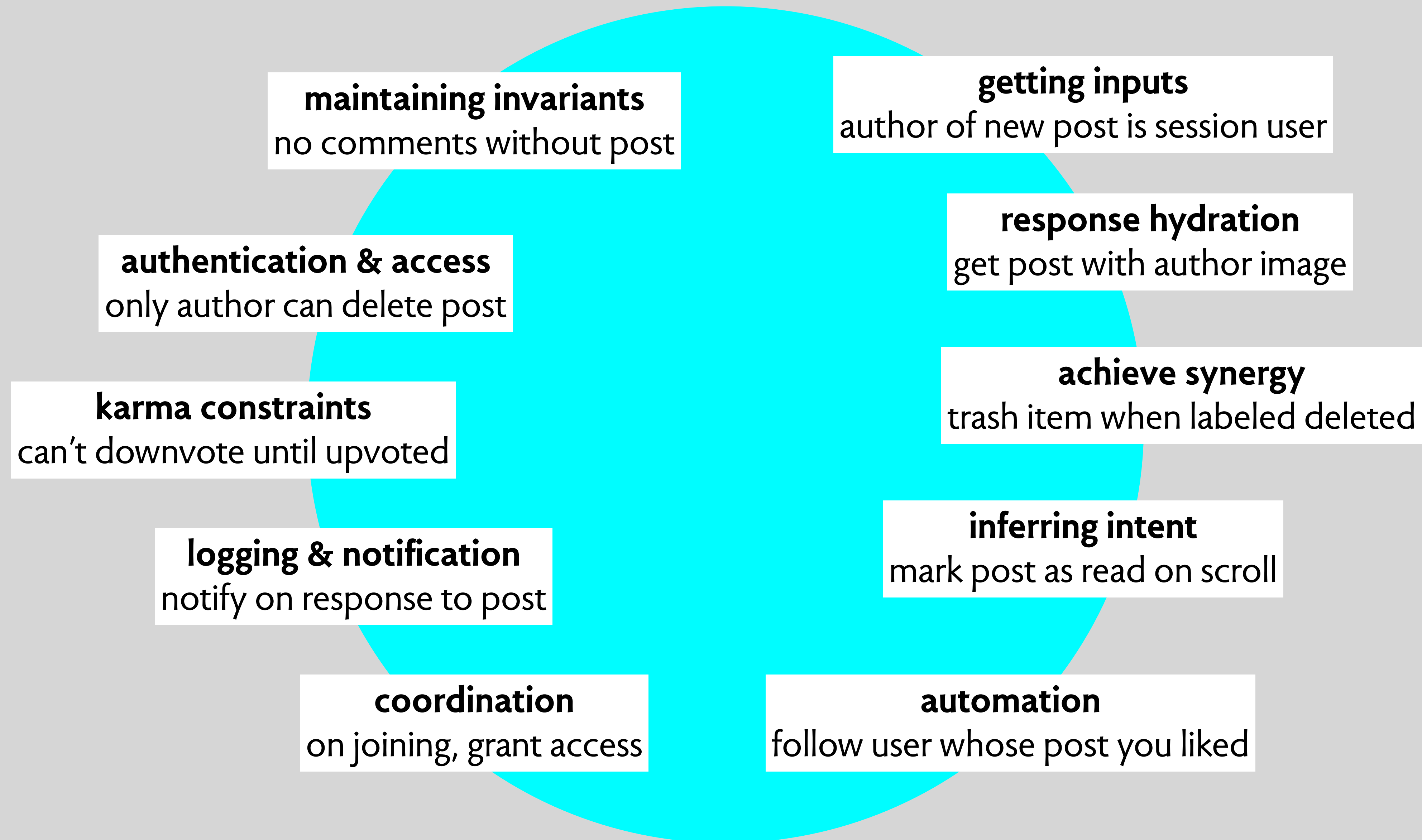


CPNWVK

<https://class.mit-sdg.dev/join>

how
reactions
are used

what are reactions used for?



*an example of
a bad reaction*



Arvind Satyanarayan

November 15, 2018 at 2:04 PM

Re: TALK: Monday 11-19-2018 Kanit (Ham) Wongsuphasawat: No...

[Details](#)

Cc: seminars@csail.mit.edu, HCI-Seminar@lists.csail.mit.edu

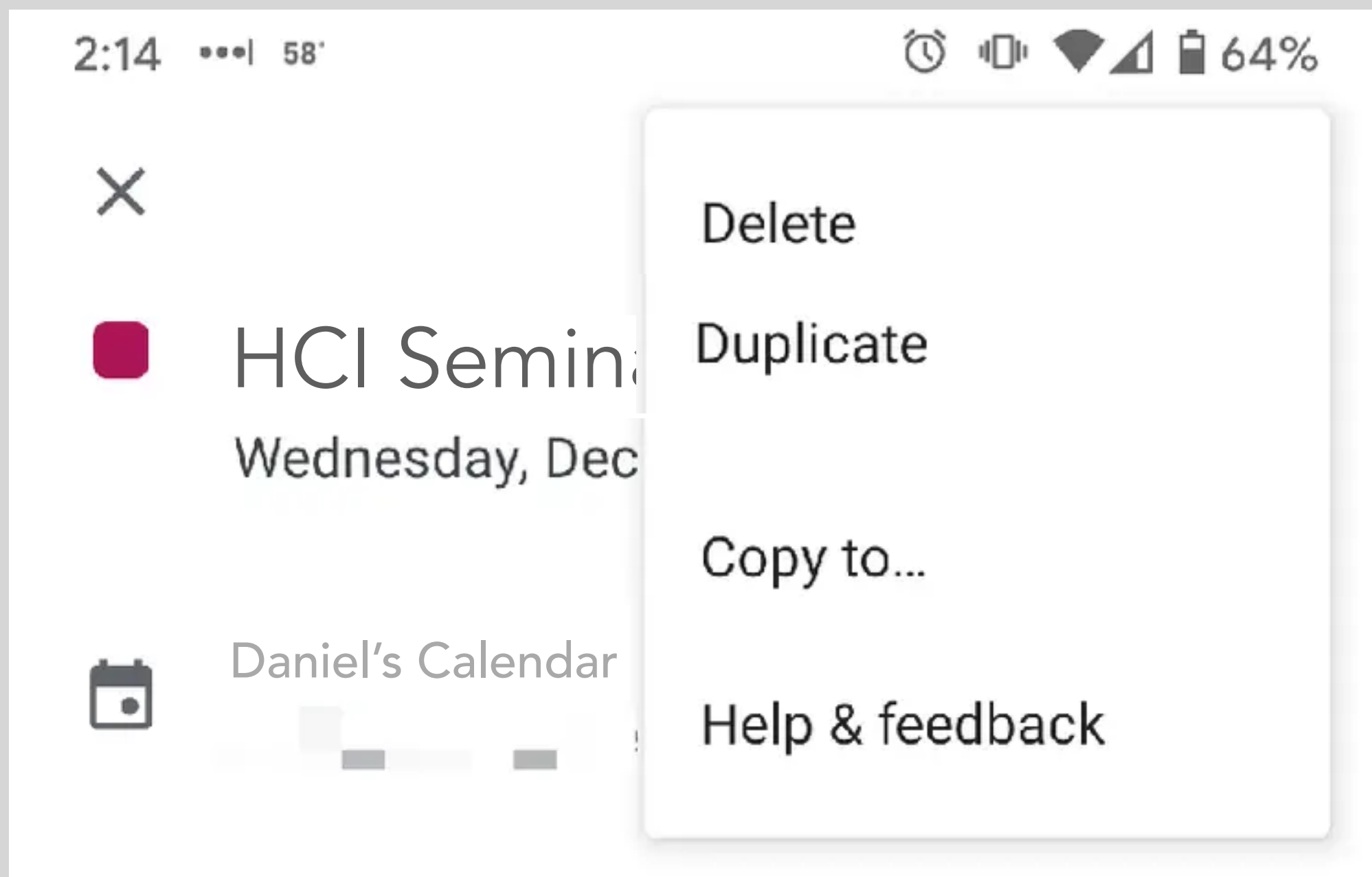


This message is from a mailing list.

[Unsubscribe](#)



Despite some erroneous messages sent to this list accidentally, Kanit's talk is happening!
Please join us on Monday.



Canceling and deleting events in the Google Calendar mobile app is similar to on a desktop.

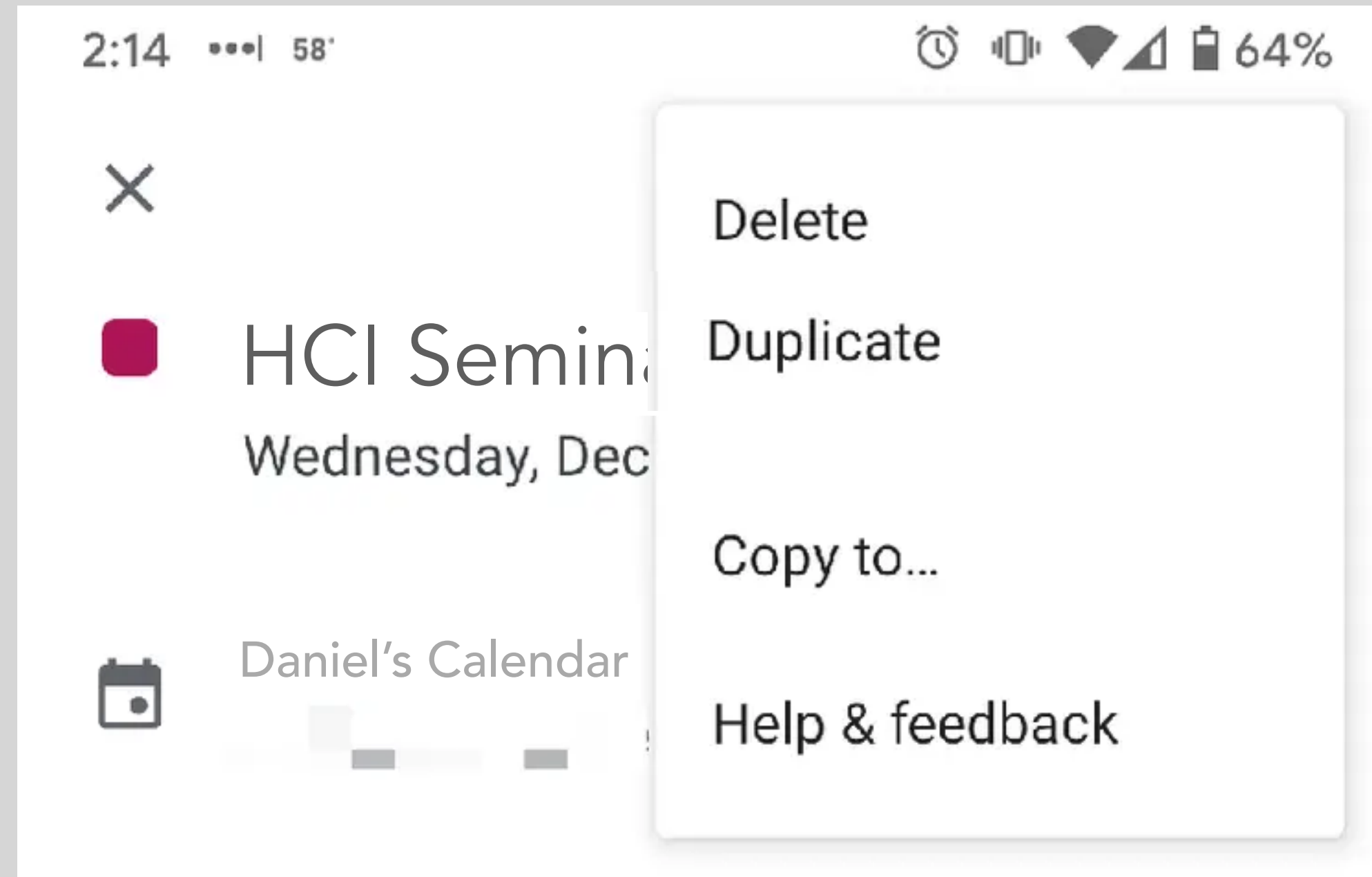
1. First, open Google Calendar.
2. Tap on the event you wish to cancel.
3. Press on the three dots in the top right corner of the event window.
4. Select Delete.
5. Tap Delete event. Google Calendar will send a cancellation email to the guests.

Mar 22, 2021

<https://wpamelia.com> › Blog

[How to Cancel an Event in Google Calendar - Amelia booking ...](#)





concept EventPlanning

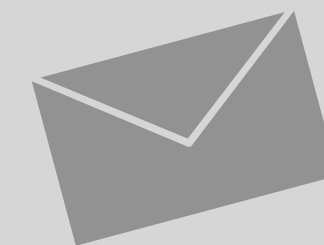
purpose record upcoming engagements

actions

create an event

delete an event

...



concept Inviting

purpose ensure mutual consent

actions

accept invitation

decline invitation

...



Are you sure you want to delete this event?

Deleting this meeting will remove it from your calendar and notify the invitees that this event has been deleted. You can't undo this action.

Cancel

Delete

a long time problem in iCal too
how to delete spam calendar events?



Are you sure you want to delete this event?

Deleting this event will notify the organizer that you're declining the event and deleting it from your calendar. You can't undo this action.

Cancel

Delete and Don't Notify

Delete and Notify

resolution to design problem
make reaction optional

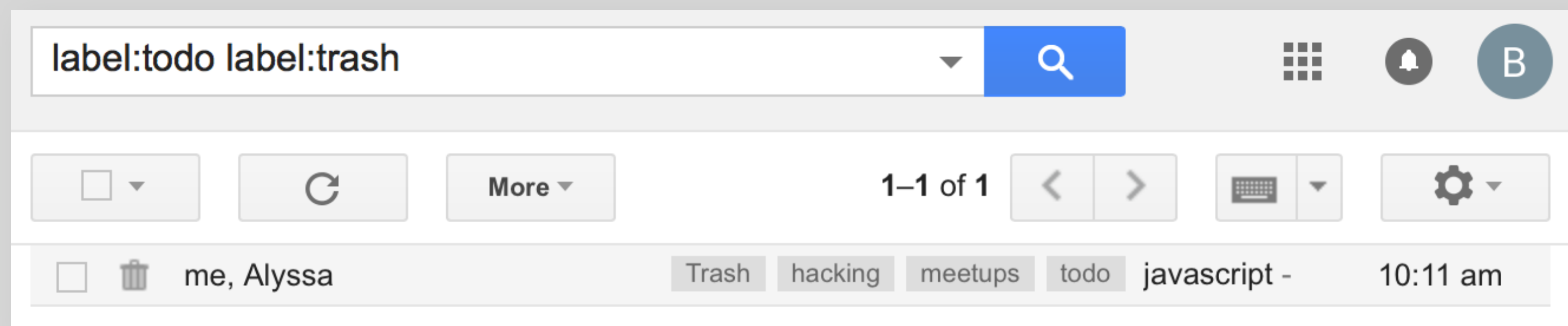
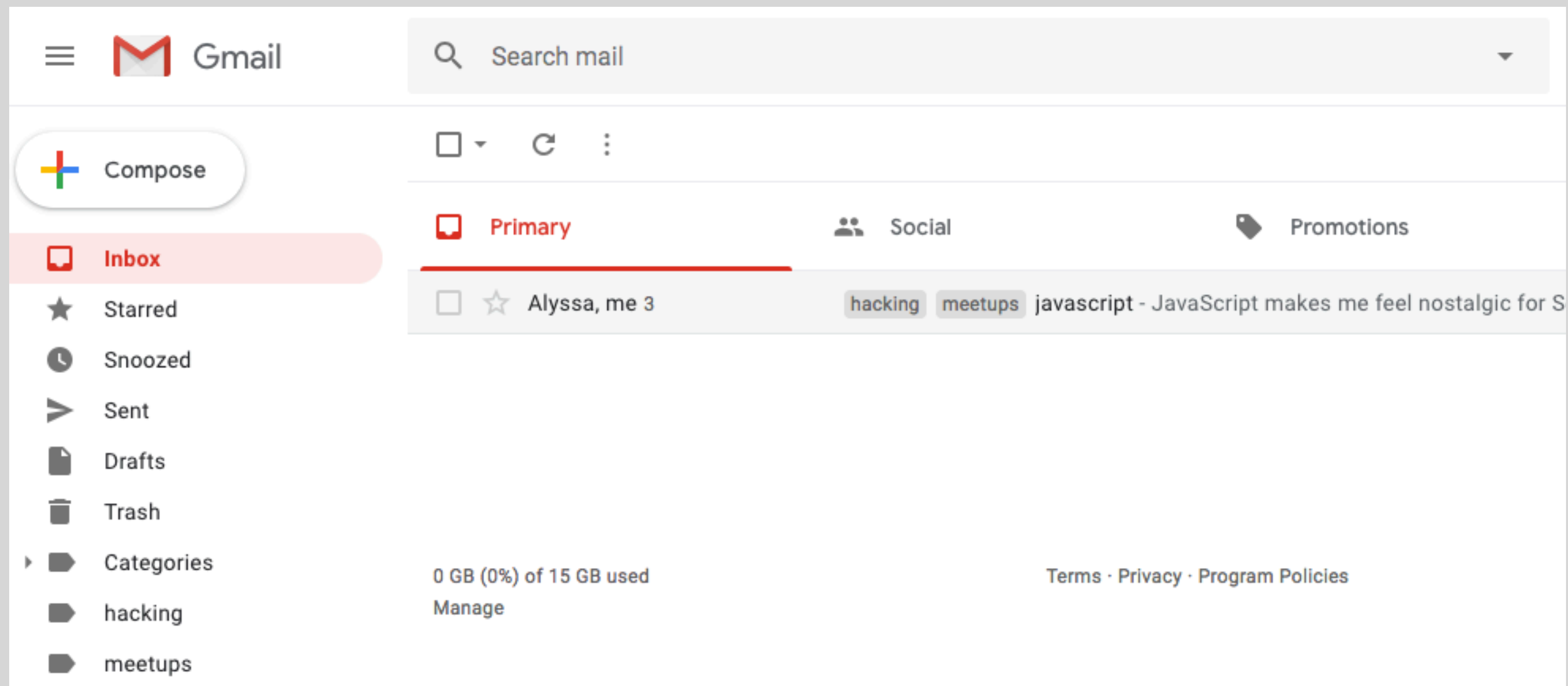
*an example of
a good reaction*

synergy in design

why is this a great design?

just a label lookup

use sent, trash in filters

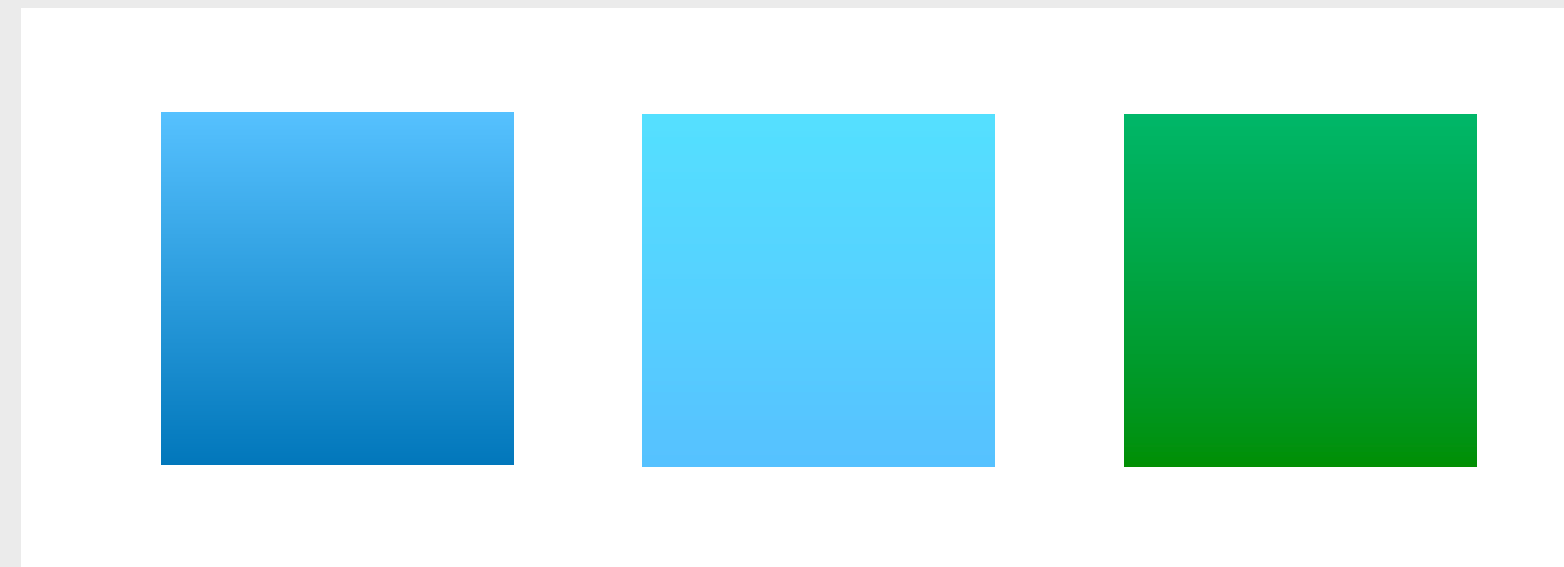


seeking
modularity

two criteria

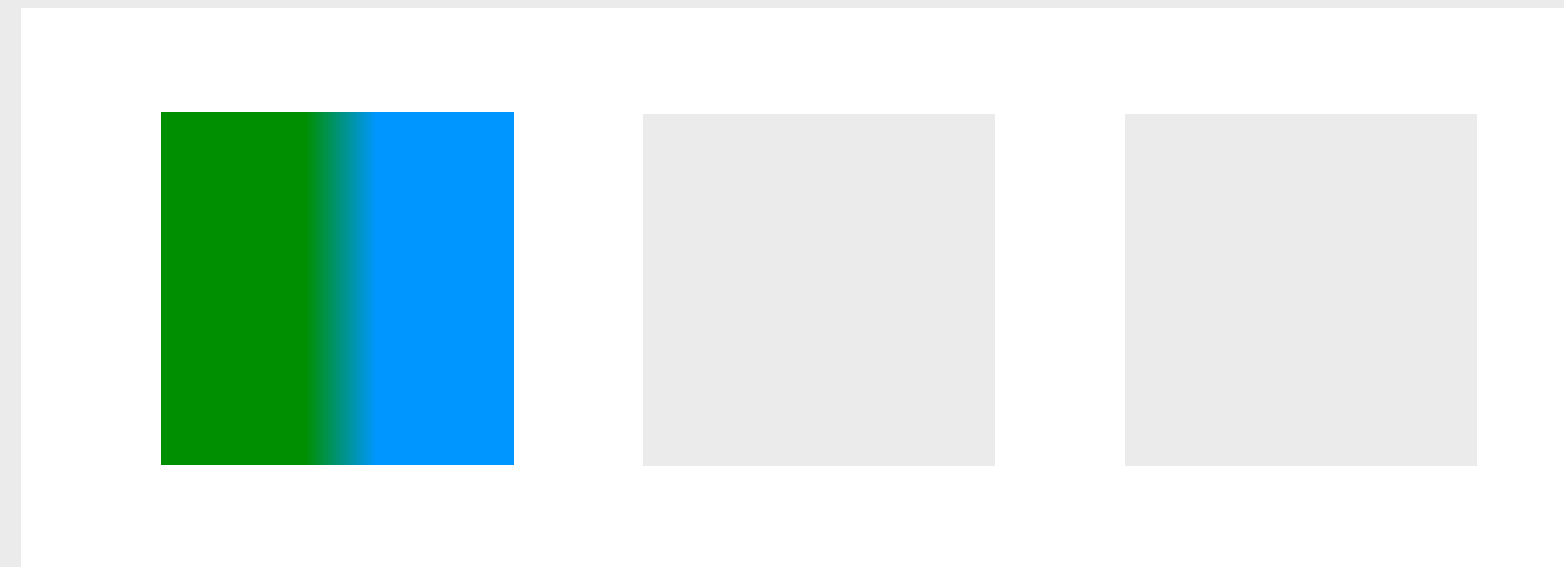
seeking criteria

imagine a division of functionality into parts
what makes this division modular?



separation vs conflation

unrelated functions are **separated**
not **conflated** in the same part

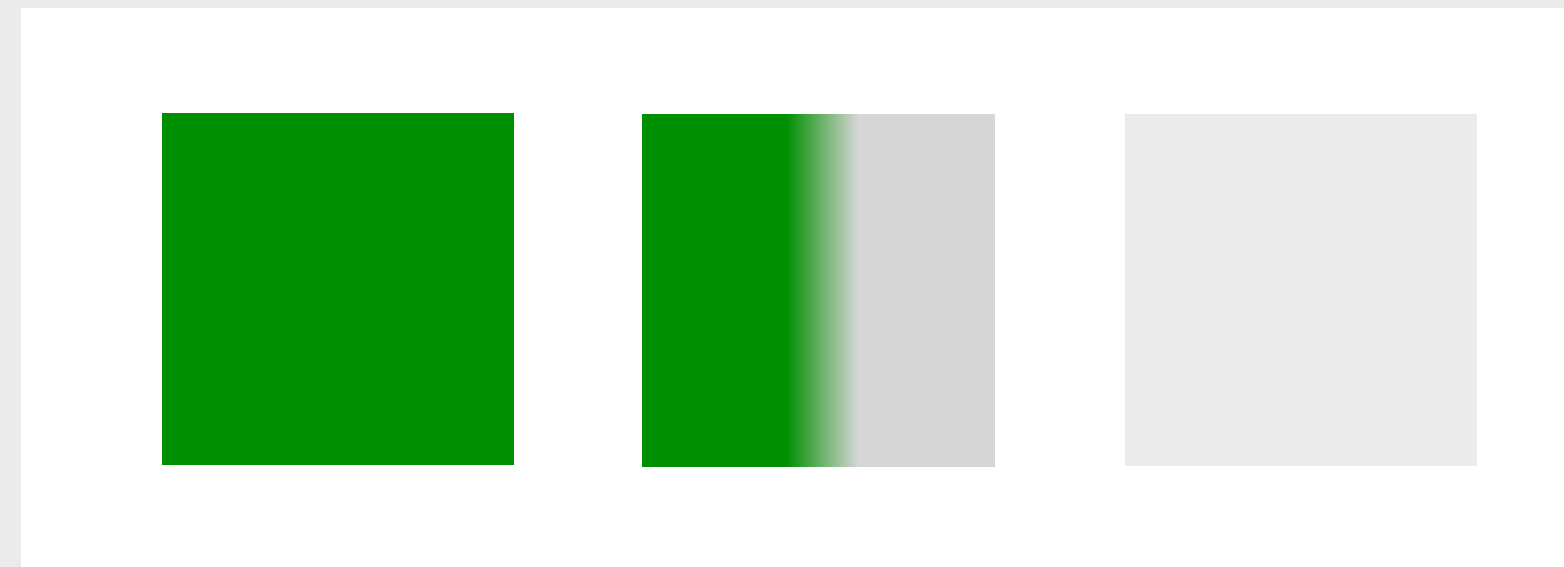


what goes wrong?
part is more complex
and less reusable

encapsulation vs fragmentation

each piece of functionality is **encapsulated**

not **fragmented** across parts



what goes wrong?
parts become coupled
changes not localized

STRATEGIES: ► CHRYSLER ► INTEL ► HEINZ

BusinessWeek

SEPTEMBER 30, 1991

A MCGRAW-HILL PUBLICATION

\$2.50

SOFTWARE MADE SIMPLE

It's called object-oriented programming—a way to make computers a lot easier to use. Here's what it can do for you.

PAGE 92



let's code
hacker news!

Y

Hacker News

new | past | comments | ask | show | jobs | submit

login

▲ Jackson structured programming (wikipedia.org)

106 points by haakonhr 63 days ago | hide | past | favorite | 69 comments

Posting

▲ danielnicholas 63 days ago [-]

user: danielnicholas

created: 63 days ago

karma: 11

Authenticating

Upvoting

Favoriting

Commenting

Karma Tracking

you might find helpful an annotated version [0] of Hoare's explanation of JSP that I edited for a Michael Jackson festschrift

I'd point to these ideas as worth knowing:

...ing problem that involves traversing ...res can be solved very systematically. HTDP addresses this class, but bases code structure only on input structure; JSP synthesized i

- The ... problems that, however you code, can't be pushed under the rug—most notably structure clashes—and just recognizing them
- Coroutines (or code transformation) let you structure code more cleanly when you need to read or write more than one structure. It's why real iterators (with yield), which offer a limited form of this, are (in my view) better than Java-style iterators with a next method.
- The idea of viewing a system as a collection of asynchronous processes (Ch. 11 in the JSP book, which later became JSD) with a long-running process for each real-world entity. This was a notable contrast to OOP, and led to a strategy (seeing a resurgence with event storming for DDD) that began with events rather than objects.

[0] <https://groups.csail.mit.edu/sdg/pubs/2009/hoare-jsp-3-29-09...>

▲ ob-nix 63 days ago [-]

... this brings back memories! In the late eighties I, as a teenager, found a Jackson Struct. Pr. book at the town library. I remember I was amazed at the text and wondered why I hadn't heard about the method before.

If I remember correctly did the book clearly point out backtracking as a standard method, while mentioning that most languages lacked that, so it had to be implemented manually.

let's build it!

```
class User {  
  String name;  
  String password;  
  User new (n, p) { ... }  
  User authenticate (n, p) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Post new (a, b) { ... }  
}
```


adding upvoting

```
class User {  
  String name;  
  String password;  
  User register (n, p) { ... }  
  User authenticate (n, p) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Set [User] ups, downs;  
  Post new (a, b) { ... }  
  upvote (u) { ... }  
  downvote (u) { ... }  
}
```

adding karma

```
class User {  
  String name;  
  String password;  
  int karma;  
  User register (n, p) { ... }  
  User authenticate (n, p) { ... }  
  award (i) { ... }  
  bool hasKarma (i) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Set [User] ups, downs;  
  Post new (a, b) {...}  
  upvote (u) {  
    author.award (10)  
  }  
  downvote (u) {  
    if u.hasKarma (10) ...  
  }  
}
```


adding commenting

```
class User {  
  String name;  
  String password;  
  int karma;  
  User register (n, p) { ... }  
  User authenticate (n, p) { ... }  
  award (i) { ... }  
  bool hasKarma (i) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Set [User] ups, downs;  
  Set [Comment] comments;  
  Post new (a, b) { ... }  
  upvote (u) {  
    author.award (10)  
  }  
  downvote (u) {  
    if u.hasKarma (20) ... }  
  addComment (c, b) { ... }  
}
```

what's wrong with this code?

```
class User {  
  String name;  
  String password;  
  int karma;  
  User register (n, p) { ... }  
  User authenticate (n, p) { ... }  
  award (i) { ... }  
  bool hasKarma (i) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Set [User] ups, downs;  
  Seq [Comment] comments;  
  Post new (a, b) { ... }  
  upvote (u) {  
    author.award (10) }  
  downvote (u) {  
    if u.hasKarma (20) ... }  
  addComment (c, b) { ... }  
}
```

User authentication

Posting

Upvoting

Commenting

Karma

conflation

Post conflates posting, karma, etc



classes are not reusable

Post class won't work in an app that doesn't have karma points

fragmentation

commenting is in Post and Comment



can't be built independently

to build Post class, need User class to have been built already

actually, it was even worse than this

and this is truly broken!

typically add a class with a name like Users

```
class User {  
  String name;  
  String password;  
  User new (n, p) { ... }  
  User authenticate (n, p) { ... }  
}
```

```
class Post {  
  User author;  
  String body;  
  Post new (a, b) { ... }  
}
```

where does the new post go?

posts might go in User: yet more needless coupling

a better way

concept Authenticating
state

a set of Users with
a username String
a password String

actions

register (username, password): (user)
authenticate (username, password): (user)

concept Upvoting [Voter, Item]
state

a set of Votes with
a Voter
an Item

actions

upvote (voter, item)
downvote (voter, item)

note the generic roles

Voter, Author, Commenter, Actor...
will all be Authentication.User

concept Posting [Author]
state

a set of Posts with
an author User
a text String

actions

addPost (author, text): (post)

concept KarmaTracking [Actor]
state

a set of Actors with
a karma Int

actions

award (actor, karma)

when

Upvoting.upvote (voter, item)

where

Posting: item has author

do

KarmaTracking.award (author, 10)

concept Commenting [Commenter, Item]

a set of Comment with
a Commenter
a target Item
a text String

actions

comment (commenter, target, text): (comment)

when

Requesting.request (downvote, user, item)

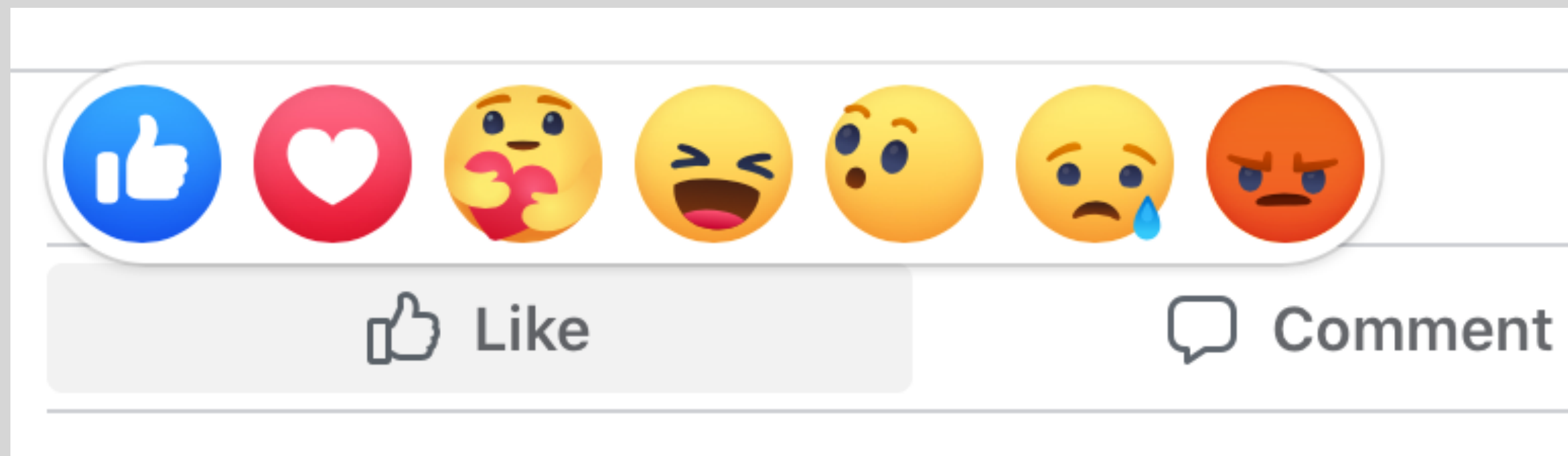
where

KarmaTracking: user has 20 or more

then

Upvoting.downvote (voter: user, item)

parting
thought



Remember our discussion about Facebook reactions, and whether this was Upvoting, Reacting or Recommending?

Can you now recast this design analysis in terms of reactions?

What are the benefits and costs to the user of the automation that happens here?

join the discussion
on the forum